

Weighted LCA with Logistic Profiling: Equations and Code

Kevin Linares

2026-06-09

Companion to `lca_profiling_workflow.qmd`. Section 1 fixes notation. Section 2 states each equation, what it is for, and where the code computes it. Section 3 walks through the code. Section 4 maps equations to code. Section 5 states what the workflow licenses. Section 6 covers running, dependencies, and adapting to real data.

Code blocks here are explanatory and not executed on render (`eval: false`); the workflow file is the script that runs.

1 Notation

Symbol	Meaning
$i; h; N_h, n_h$	respondent; stratum; stratum population and sample sizes
w_i	Horvitz–Thompson design weight, N_h/n_h
$k = 1, \dots, K; k^*$	latent classes; the designated target class
$j; C_j; y_{ij}$	item; its category count; respondent i 's answer
$\mathbf{x}_i$	profiling covariates (<code>age_z</code> , <code>ideology_z</code> , <code>college</code>), disjoint from the items
$C_i; \hat{C}_i$	true class; modal assignment
$\pi_k; \rho_{jck}; p_{ik}$	class prevalence; response probability; posterior probability
γ	coefficients of the profiling logistic regression

In code: `samp` (data), `best_w` (weighted LCA fit), `post/modal` (posteriors and assignments), `D` (classification table), `des` (the `svydesign` object), `fit_modal/fit_oracle` (the `svyglm` fits), `Q/U/qbar/Tv` (pseudo-class draw machinery), `coef_tbl` (the comparison table).

2 Equations

2.1 Horvitz–Thompson design weight

$$w_i = \frac{1}{\pi_i} = \frac{N_h}{n_h} \quad \text{under stratified SRS.}$$

Each respondent stands in for w_i population members. Weighting is what makes sample quantities estimate population quantities under the design; the strata with large weights are the under-sampled ones.

In code. `w = N_h[stratum] / n_h` in `sim-draw`; the same weights enter the EM and the `svydesign` object.

2.2 The design-weighted latent class model

$$\ell_w(\theta) = \sum_i w_i \log \sum_{k=1}^K \pi_k \prod_j \rho_{j y_{ij} k},$$

maximized by an EM whose E-step computes the posteriors

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_m \pi_m \prod_j \rho_{j y_{ij} m}}$$

and whose M-step is closed-form weighted proportions,

$$\hat{\pi}_k = \frac{\sum_i w_i p_{ik}}{\sum_i w_i}, \quad \hat{\rho}_{jck} = \frac{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} = c)}{\sum_i w_i p_{ik}}.$$

The maximizers are pseudo-maximum-likelihood estimates of the finite-population parameters. The weights enter only the M-step; the posteriors are per-respondent conditional probabilities. Numerically, the E-step runs on the log scale and is normalized by log-sum-exp, and ρ is floored before logs.

In code. `em_run` in `em-core`. The E-step: per-item log response probabilities indexed by the observed answers (`map2(state$rho, Y_cols, ~ log(.x)[.y, ])`), summed by `reduce`, plus log prior; `rowLogSumExps`; `exp(logdens - lse)`. The M-step: `wp <- w * post`, class sizes `colSums(wp)` normalized, response probabilities `crossprod(onehot, wp)` column-normalized with a `1e-12` floor. Iteration is a `reduce` fold; no loops anywhere in the file.

2.3 Identification: starts and alignment

$$\sigma^* = \arg \min_{\sigma \in S_K} \sum_k \|\rho_{\sigma(k)}^{\text{fit}} - \rho_k^{\text{ref}}\|_2^2.$$

Class labels are arbitrary and the likelihood has local maxima, so the model is fit from multiple random starts and the best fit's classes are relabeled by the permutation minimizing the squared distance between class response profiles, solved exactly by the Hungarian algorithm.

In code. `align_to` with `clue::solve_LSAP`; twelve starts in `fit-lca`, best by weighted log-likelihood.

2.4 Classification quality: entropy and the D table

$$E = 1 - \frac{-\sum_i \sum_k p_{ik} \ln p_{ik}}{n \ln K}, \quad D_{ks} = \frac{\sum_{i: \hat{C}_i=k} w_i p_{is}}{\sum_{i: \hat{C}_i=k} w_i}.$$

Entropy summarizes separation (one is perfect). The classification table D gives, for each modally assigned class k , the weighted average posterior mass on every class s ; its off-diagonal entries are the classification error rates that attenuate the modal regression and that the maximum likelihood three-step correction takes as input. Both numbers are part of the deliverable, because they tell the reader how much the modal regression understates the profile.

In code. `classify` chunk: `ent` from `post`; D via `rowsum(w * post, modal)` divided by `rowsum(w, modal)`.

2.5 The profiling regression

$$\text{logit } P(C_i = k^* \mid \mathbf{x}_i) = \gamma_0 + \gamma_1 \text{age}_i + \gamma_2 \text{ideology}_i + \gamma_3 \text{college}_i,$$

estimated three ways on the same `svydesign` object:

1. **Modal:** outcome $\mathbf{1}(\hat{C}_i = k^*)$. Design-weighted point estimates; linearization (Taylor series) standard errors that respect strata, clusters, and weights. Attenuated toward zero by classification error (Bolck, Croon, and Hagenaars 2004).
2. **Pseudo-class draws:** outcome $\mathbf{1}(\tilde{C}_i^{(m)} = k^*)$ with $\tilde{C}_i^{(m)}$ drawn from the posterior, repeated M times and pooled by Rubin's rules,

$$\bar{\gamma} = \frac{1}{M} \sum_m \hat{\gamma}^{(m)}, \quad T = \bar{U} + \left(1 + \frac{1}{M}\right) B,$$

with $\bar{U}$ the mean within-draw variance and B the between-draw variance of the coefficients. This widens the intervals by the assignment uncertainty; it does not undo the attenuation.

3. **Oracle** (simulation only): outcome $\mathbf{1}(C_i = k^*)$ with the true class. The benchmark; the modal-to-oracle coefficient ratio is the measured attenuation.

`family = quasibinomial()` is used in `svyglm` because the weighted estimating equations are quasi-likelihood, not a literal binomial likelihood; it is the standard and warning-free choice for design-based logistic regression.

In code. `profile` chunk: `des <- svydesign(ids = ~psu, strata = ~stratum, weights = ~w, nest = TRUE); fit_modal, fit_oracle`; the draw loop is a `map` over `M_draws`, each iteration adding a drawn outcome with `update(des, y_draw = ...)` and refitting; pooling in `qbar`, `Bv`, `Tv`.

2.6 The deliverable curve

$$\hat{P}(C = k^* \mid \text{age, college}) = \text{logit}^{-1}(\hat{\gamma}_0 + \hat{\gamma}_1 \text{age} + \hat{\gamma}_3 \text{college}), \quad \text{ideology} = 0,$$

the predicted membership probability across the covariate space, which is the form a messaging application consumes: where the target class concentrates, not just which coefficients are positive.

In code. `fig-profile`: a prediction grid through `predict(fit_modal, newdata, type = "response")`, with weighted age deciles as a rug.

3 Code walkthrough

No `for` or `while` anywhere; iteration is `purrr` and matrix algebra.

3.1 Simulation (`sim-params`, `sim-draw`)

`beta` holds the true multinomial coefficients linking the covariates to class membership; `class_probs()` is the softmax. The stratum mean of `age_z` rises across strata while the design weight falls, which makes selection informative. PSU random intercepts induce intra-cluster correlation, which is why the `svydesign` object declares the PSUs. Items are drawn from the class through `true_rho`, built with a moderate separation constant (0.9) so classification is realistic rather than near-perfect. The estimand `pop_pi` integrates the generative model over each stratum's covariate distribution by Monte Carlo (20,000 draws per stratum) and weights strata by `N_h`.

`draw_cat_rows` and `draw_item` are vectorized inverse-CDF categorical samplers: row-wise cumulative probabilities by `rowCumsums`, one uniform per row, count of cumulative sums below the uniform.

3.2 Stage one (`em-core`, `fit-lca`)

As in section 2.2. The two unweighted lines exist only to show the prevalence bias in Table 1; everything downstream uses the weighted fit `best_w`. The `stopifnot` guards that every response-probability column sums to one.

3.3 Classification (`classify`)

`modal` is the row-wise argmax of the posterior matrix. `D` is computed by grouped weighted row sums: `rowsum(w * post, modal)` gives, for each assigned class, the weighted posterior mass on every class; divided by the assigned-class weight totals it is the weighted mean posterior by assignment.

3.4 Profiling (`profile`)

The design object is built once, with the modal and true outcome indicators added as columns. `svyglm(..., family = quasibinomial())` fits the design-based logistic regressions. The pseudo-class block draws a fresh assignment vector per iteration with the same `draw_cat_rows` sampler applied to the posterior matrix, attaches it to the design with `update`, refits, and stores coefficients and variance diagonals; `reduce(rbind)` stacks them and three lines implement Rubin's rules. Twenty-five draws are enough for stable pooled variances at this scale.

3.5 Tables and figures

`coef_tbl` assembles the three estimators in long form with normal-approximation 95 percent intervals (pooled intervals use the Rubin total variance; a refined degrees-of-freedom correction is omitted and noted). `fig-forest` is the comparison plot; `fig-profile` is the application deliverable; `tbl-dmatrix` prints `D`. Tables use `kable(..., booktabs = TRUE)` for the PDF.

4 Equation to code map

Equation	Section	Code object	Chunk
HT weight	2.1	<code>w</code>	<code>sim-draw</code>
Weighted pseudo-log-likelihood, EM	2.2	<code>em_run</code>	<code>em-core</code> , <code>fit-lca</code>
Alignment	2.3	<code>align_to</code>	<code>em-core</code>
Entropy; <code>D</code> table	2.4	<code>ent</code> ; <code>D</code>	<code>classify</code>

Equation	Section	Code object	Chunk
Profiling logit, three estimators	2.5	<code>fit_modal,</code> <code>draw_fits,</code> <code>fit_oracle</code>	<code>profile</code>
Rubin pooling	2.5	<code>qbar, Bv, Tv</code>	<code>profile</code>
Predicted-probability curve	2.6	<code>grid, predict</code>	<code>fig-profile</code>

5 What the workflow licenses

Licensed. A population-correct typology (weighted prevalences and measurement profiles); a design-based description of who is in the target class, as coefficients with design-correct standard errors and as predicted-probability curves; and an explicit statement of classification quality (entropy, the D table) so the reader can judge the attenuation. With high classification quality, the modal regression reads as a conservative lower bound on the covariate gradients.

Not licensed. Unattenuated magnitudes when classification quality is poor. The corrections for that regime operate on exactly this parametric third step: the BCH adjustment and the maximum likelihood three-step method (Vermunt 2010; Bakk, Tekle, and Vermunt 2013), the two-step estimator (Bakk and Kuha 2018), or a one-step latent class regression (Bandein-Roche et al. 1997). In R terms, the maximum likelihood correction refits the latent class model with the modal assignment as a single error-prone indicator whose error matrix is fixed at D; this is an extension of the existing EM, not a new estimator. The pseudo-class pooling here is uncertainty propagation, not bias repair.

Standing cautions. Profiling covariates disjoint from the items; weights remove only the selection the weights encode (representation, in total survey error terms); enumeration by weighted BIC precedes everything in applied use; one simulated draw demonstrates mechanics, not sampling properties.

6 Running it, dependencies, and outputs

Dependencies. `tidyverse`, `matrixStats`, `survey`, `clue`, `viridisLite`; Quarto with a LaTeX distribution for the PDF (figures render through `cairo_pdf`).

Run. `quarto render lca_profiling_workflow.qmd`. Chunk order is load-bearing: simulation, EM, classification, design object, regressions, summaries, tables, figures.

Outputs. `lca_profiling_caselevel.csv` (id, design variables, posteriors, modal class) and `lca_profiling_coefficients.csv` (the three-estimator comparison), both written silently on render.

Adapting to real data. Replace the simulation with your data frame; set `cats` and `item_names` to your battery; set `pred_names` to your demographics and opinions, verifying they share nothing with the battery; declare your real strata, PSUs, and weights in `svydesign` (or use `svrepdesign` if your file ships replicate weights, in which case `svyglm` works unchanged); pick `target` substantively; choose K by weighted BIC first. Drop the oracle regression, which exists only because the simulation knows the truth, and report the modal and pooled-draw estimates together with the entropy and D table.